

ATTRIBUTION-PATCHING-GUIDED REPLAY REFINEMENT FOR TASK-VECTOR MERGING

Anonymous authors

Paper under double-blind review

ABSTRACT

We study the task of merging fine-tuned variants of a shared pretrained model when a small labeled replay buffer for each task is available. Starting from any initial model—a merge produced by any existing method, or the pretrained model itself—we refine it with task gradients while using the known task experts as coordinate-wise trust anchors. The resulting update admits two equivalent motivations that we develop in parallel. As a parameter-space analogue of attribution patching, the effect of moving the current model toward a task expert is approximated by a first-order loss attribution, and only coordinates for which the expert direction is locally loss-decreasing are used. As an expert-anchored trust-region replay refinement, the departure from ordinary replay gradient descent is that each coordinate’s step is weighted by its distance to the expert, gated to keep only steps that move toward the expert, and clipped so it cannot overshoot—the same coordinates that the attribution view selects. Both perspectives lead to a single refinement algorithm, which is provably non-expansive toward the experts’ box hull at any learning rate. We evaluate it on multi-head GLUE-8, the eight- and twenty-task CLIP suites, and a decoder-only MergeBench track, under a protocol in which every hyperparameter is selected on a small held-out buffer and the evaluation split is never used for tuning, at a budget of sixteen labeled examples per task or fewer. Refining directly from the pretrained model, the anchored update is the best-performing treatment on every benchmark, while unconstrained replay descent is not a viable cold-start method at this budget; from a merge initialization it improves every starting point tried and makes the outcome far less sensitive to which merge it was handed. Its advantage widens as the buffer shrinks: at eight labeled examples per task it still beats a tuned task-arithmetic merge from the pretrained model alone, in a regime where unconstrained descent overfits a quarter of its configurations while its replay loss still improves. On held-out tasks never merged or refined, the refinement populates the accuracy–retention Pareto frontier, and every checkpoint-only merge is dominated.

1 INTRODUCTION

Merging fine-tuned variants of a shared pretrained model combines their skills in weight space, without retraining, ensembling, or routing. But merges degrade under task interference, and the data-free repairs—sparsification, sign election, learned coefficients—plateau well below the experts. In many practical settings a small *labeled* replay buffer per task is available (it is routinely retained for validation or continual learning); the question is how to spend such a tiny labeled budget so that it repairs the merge without giving up what made merging attractive: a single model, low cost, and bounded drift from checkpoints one already trusts.

We refine the merged model with replay gradients *anchored to the task experts*: each coordinate’s step is kept only where moving toward the expert locally reduces that task’s replay loss (an attribution-patching sign), weighted by the coordinate’s distance to the expert, and clipped so it can never overshoot—applied task-sequentially, from *any* initialization, including the pretrained model itself. The update is provably non-expansive toward the experts’ box hull at any learning rate (Proposition 1).

Our contributions are: (i) a replay-refinement method with two equivalent derivations—parameter-space attribution patching and an expert-anchored trust region—and a divergence-safety guarantee;

(ii) a uniform evaluation across four benchmark families (multi-head GLUE-8, the eight- and twenty-task CLIP suites, and decoder-only LLM merging in the MergeBench setting) under a strict protocol in which every hyperparameter is chosen on a small held-out selection buffer and the evaluation split is never used for tuning, with matched data budgets and sanity controls; (iii) evidence that from the pretrained model—the setting with no merge to inherit—the anchored update is the only one of the three replay families that turns a 16-example buffer into multi-task ability, while from a merge initialization it improves every starting point tried and makes the outcome markedly less sensitive to which merge was handed to it; (iv) a characterization of where the anchor earns its keep: as the buffer shrinks to eight examples per task, unconstrained descent silently overfits—a quarter of its configurations lose test accuracy while their replay loss is still falling—whereas the trust region removes that failure mode entirely, and no anchored run fell below its initialization at any setting tested; and (v) a measured accuracy–retention frontier on tasks never merged or refined, populated by the refinement together with the strongest data-fitted merge, on which every checkpoint-only merge and both unconstrained controls are dominated.

2 RELATED WORK AND BACKGROUND

Model merging and task vectors. Model merging combines fine-tuned variants of a shared pretrained model into a single model in weight space, avoiding ensembling overhead: model soups average compatible checkpoints (Wortsman et al., 2022), Fisher merging weights parameters by Fisher information (Matena and Raffel, 2022), and task arithmetic represents fine-tuning as a *task vector* $\tau_t = \theta_t - \theta_0$ and merges as $\theta_0 + \sum_t \lambda_t \tau_t$ (Ilharco et al., 2022)—checkpoint-only, but prone to over-counted directions and sign or magnitude conflicts across tasks.

Interference-aware and data-dependent merging. A major line of work addresses interference between task vectors: TIES-Merging trims small-magnitude updates and merges only sign-consistent components (Yadav et al., 2023); DARE randomly drops most delta parameters and rescales the rest (Yu et al., 2023), generalized by DELLA (Deep et al., 2024); Model Breadcrumbs removes negligible and outlier perturbations (Davari and Belilovsky, 2023); Localize-and-Stitch stitches sparse task-relevant regions back into the pretrained model (He et al., 2024). Most closely related to our trust-region view, Task Arithmetic in Trust Region (TATR) uses task-loss gradients at the pretrained model to identify gradient-orthogonal parameter dimensions in which task vectors can be merged with limited interference (Sun et al., 2025). TATR is training-free and restricts the merge itself; our method instead uses labeled replay gradients at the evolving merged model to take sequential, expert-directed refinement steps. A complementary line uses small amounts of data or model outputs: APL scores task vectors by a data-dependent contrast (Kong et al., 2024), RegMean solves each linear layer by regression on its inputs (Jin et al., 2023), and AdaMerging learns coefficients by entropy minimization (Yang et al., 2024). Twin-Merging, Task Vector Bases, and FusionBench extend and standardize this space (Lu et al., 2024; Zeng et al., 2025; Tang et al., 2024). The present work does not introduce another pruning rule: it asks whether a small *labeled* replay buffer can repair an initial merge while staying close to the known experts, using the sign of a merge-to-expert attribution only as a local gate for refinement.

Attribution patching. Attribution patching approximates the effect of an intervention by a first-order Taylor expansion, replacing many explicit interventions with gradient-based scores (Syed et al., 2024; Kramar et al., 2024); in parameter space the analogue is a directional derivative of the task loss along a chosen displacement—here, the movement from the current merged model toward a task expert. Extended positioning against the benchmark families of the merging literature is given in Appendix C.

3 METHOD

3.1 PROBLEM SETUP

Let $\mathcal{T} = \{1, \dots, T\}$ be a set of tasks whose experts are all initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$; fine-tuning on task t gives expert parameters θ_t and task vector $\tau_t = \theta_t - \theta_0$. Writing $r \in \{1, \dots, d\}$ for a parameter coordinate, the refinement starts from an *arbitrary* initial

model $\theta^{(0)} \in \mathbb{R}^d$: any weight-space merge of the experts—task arithmetic

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^T \lambda_i \tau_i \quad (1)$$

with fixed coefficients λ_i is the simplest instance, but TIES, RegMean, or AdaMerging merges serve equally (Section 5), as does the pretrained model itself, $\theta^{(0)} = \theta_0$ (the $\lambda_i = 0$ case; Section 5.1). Nothing in the method depends on how $\theta^{(0)}$ was constructed; only the experts $\{\theta_i\}$ enter the update. Let $\mathcal{D}_t^{\text{probe}}$ be a small labeled replay buffer used for refinement, $\mathcal{D}_t^{\text{eval}}$ the evaluation split used only for reporting, and $\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}})$ the replay-buffer loss of encoder parameters θ under task i 's prediction head. In the multi-head settings task-specific heads are not merged and the merged encoder is evaluated with each task's own head; the shared-output settings merge everything. The method uses no base-relative saliency term and no pre-merge pruning mask: its only data-dependent operation is a short replay-buffer refinement of $\theta^{(0)}$.

3.2 TRUST-REGION REPLAY REFINEMENT

The natural data-dependent baseline from this initialization is ordinary replay-buffer gradient descent, $\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^T \nabla_{\theta} \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}})$, which treats the merge like any other starting point and lets the replay gradients move every coordinate freely, with nothing tying the refined model to the known experts. The central departure of our method is expert anchoring: for each task i and coordinate r we (i) *weight* the negative-gradient step by the current distance $|v_{i,r}|$ to the expert, so coordinates already close to the expert barely move; (ii) *gate* the step, keeping only coordinates where it moves the model *toward* the expert; and (iii) *clip* the result to the expert-distance trust region, so a single update can never overshoot the expert.

Let $\theta^{(s,0)} = \theta^{(s)}$ and let π_s be the task order for sweep s . For within-sweep index $k \in \{1, \dots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_{\theta} \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \quad (2)$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}, \quad (3)$$

$$m_{i,r}^{(s,k)} = \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}, \quad (4)$$

$$\tilde{u}_{i,r}^{(s,k)} = -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}. \quad (5)$$

The mask $m_{i,r}^{(s,k)}$ is a trust-region acceptance rule—keep only the coordinates whose replay step also moves the current model toward expert i —and its condition $g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0$ is exactly the attribution-patching sign that Section 3.3 derives in Equation (7): the two perspectives agree coordinate by coordinate. The raw update is scaled by the learning rate η and *then* clipped by the current distance to the expert,

$$u_{i,r}^{(s,k)} = \text{clip}\left(\eta \tilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|\right), \quad (6)$$

and the model is updated immediately, $\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}$, with $\theta^{(s+1)} = \theta^{(s,T)}$ after all T tasks in the sweep have been processed.

Because the clip follows the learning-rate scaling, per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η : the learning rate controls how many coordinates reach the trust-region boundary, never the maximum step. Optionally, $u_i^{(s,k)}$ is RMS-normalized tensor-wise. Algorithm 1 summarizes the procedure.

Algorithm 1 Sequential expert-direction-gated replay refinement

Require: Task experts $\{\theta_i\}_{i=1}^T$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^T$; initial model $\theta^{(0)}$ (any weight-space merge of the experts, or the pretrained model θ_0 itself); steps S ; learning rate η ; gate threshold ϵ_{gate} .

```

1: for  $s = 0, \dots, S - 1$  do
2:    $\theta^{(s,0)} \leftarrow \theta^{(s)}$ .
3:   Choose a task order  $\pi_s$  over  $\{1, \dots, T\}$ .
4:   for  $k = 1, \dots, T$  do
5:      $i \leftarrow \pi_s(k)$ .
6:      $g_i^{(s,k)} \leftarrow \nabla_{\theta} \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$  using task  $i$ 's head.
7:      $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$ .
8:     for each mergeable coordinate  $r$  do
9:        $m_{i,r}^{(s,k)} \leftarrow \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < -\epsilon_{\text{gate}}\}$ .
10:       $\tilde{u}_{i,r}^{(s,k)} \leftarrow -g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}| m_{i,r}^{(s,k)}$ .
11:       $u_{i,r}^{(s,k)} \leftarrow \text{clip}(\eta \tilde{u}_{i,r}^{(s,k)}, -|v_{i,r}^{(s,k)}|, |v_{i,r}^{(s,k)}|)$ . {clip after learning-rate scaling}
12:    end for
13:     $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$ .
14:  end for
15:   $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$ .
16: end for
17:
18: return  $\theta^{(S)}$ .
```

The informal claim that the refinement ‘‘cannot diverge’’ is made precise by the following guarantee, whose proof (as an instance of a general geometric result, Theorem 1) is given in Appendix A.

Proposition 1 (Non-expansiveness toward the expert box). *Let $B_{\text{exp}} := \prod_{r=1}^d [\min_{j \in \mathcal{T}} \theta_{j,r}, \max_{j \in \mathcal{T}} \theta_{j,r}]$ be the axis-aligned box hull of the task experts, and let $d_p(\cdot, B_{\text{exp}})$ denote ℓ^p distance to it. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every $1 \leq p \leq \infty$ and every sweep s and within-sweep index k ,*

$$d_p(\theta^{(s,k)}, B_{\text{exp}}) \leq d_p(\theta^{(s,k-1)}, B_{\text{exp}}),$$

and hence $d_p(\theta^{(S)}, B_{\text{exp}}) \leq d_p(\theta^{(0)}, B_{\text{exp}})$ after any number of sweeps, at any learning rate.

Every gated, clipped step moves each coordinate between its current value and the corresponding expert coordinate, and such movement can never increase the distance to the box; the box is moreover exactly the ℓ^1 -geodesic convex hull of the experts (Proposition 2), not an arbitrary safe region. The guarantee holds for any initialization $\theta^{(0)}$, including the pretrained model.

3.3 A SECOND PERSPECTIVE: ATTRIBUTION PATCHING IN PARAMETER SPACE

The same per-coordinate update arises from an interpretability-style argument that treats each move toward a task expert as a parameter-space intervention. Let ϑ denote the current encoder, $v_i(\vartheta) = \theta_i - \vartheta$ the merge-to-expert direction, and $g_i(\vartheta)$ the task gradient under task i 's head. The first-order attribution-patching estimate of the loss change from moving ϑ toward expert i is $\Delta_i^{\text{AP}}(\vartheta) = \langle g_i(\vartheta), v_i(\vartheta) \rangle$, whose coordinate-wise contribution $g_{i,r} v_{i,r}$ is negative exactly when moving coordinate r toward the expert is locally predicted to reduce task i 's loss. The loss-decreasing expert gate is therefore

$$m_{i,r}(\vartheta) = \mathbf{1}\{g_{i,r}(\vartheta) v_{i,r}(\vartheta) < -\epsilon_{\text{gate}}\}, \quad (7)$$

where $\epsilon_{\text{gate}} \geq 0$ is an optional confidence threshold (default 0), and the masked, distance-scaled step $\tilde{u}_{i,r} = -g_{i,r} |v_{i,r}| m_{i,r}$ recovers the update of Section 3.2: whenever the gate is on, $-g_{i,r}$ and $v_{i,r}$ share a sign, so the update points toward the expert, with a step that shrinks as the coordinate approaches it. The gate certifies only a local first-order effect for task i ; cross-task interference must be measured experimentally.

4 EXPERIMENTAL SETUP

Settings. We evaluate on four benchmark families. *Multi-head* GLUE-8 (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE) (Wang et al., 2018) with RoBERTa-base (Liu et al., 2019) merges the shared encoder and evaluates with each task’s own trained head—an intentionally *oracle-task-id* setting. A *CLIP/ViT-B/32* vision track uses the standard eight-task task-vector suite (Radford et al., 2021; Ilharco et al., 2022) and its twenty-task extension (Wang et al., 2024), probing a different modality and the high-interference regime (Section 5.1). A *decoder-only LLM* track adopts the MergeBench setting (He et al., 2025): published full-parameter domain experts share a common pretrained checkpoint, so task vectors and merge-to-expert directions are well defined and no expert training is required on our side. We use Llama-3.2-3B with the mathematics, coding, instruction-following, and multilingual experts, scored by judge-free automatic metrics, and merge the full model including the LM head—so every task decodes through one shared output and no task identity is assumed at evaluation, the assumption the multi-head setting grants for free. Refinement uses $n \leq 16$ labeled replay examples per task ($n = 8$ on MergeBench), class-balanced where possible and disjoint from both the selection buffer and evaluation; full details appear in Appendix C.

Baselines. Baselines are grouped by data regime: *checkpoint-only* (task arithmetic, TIES, DARE-TIES, Breadcrumbs); *data-dependent but label-free* (AdaMerging); *labeled* (ordinary replay GD from the same initialization, and the ungated distance-scaled update—the two ablations that isolate the anchor and the gate); and *sanity controls* (inverted and density-matched random gates). All three labeled families use the same replay examples, the same selection buffer, and the same hyperparameter search budget. Each merge baseline’s own hyperparameters (coefficient, density, sparsity) are likewise selected on the selection buffer.

Metrics. Tasks are scored by their standard validation metrics: Matthews correlation for CoLA, correlation for STS-B, matched/mismatched accuracy for MNLI, accuracy or accuracy/F1 elsewhere on GLUE, and top-1 accuracy on the CLIP suites; the MergeBench tasks use judge-free automatic metrics (verifiable instruction constraints, exact match on grade-school mathematics, and functional-correctness pass rates on code). These are not commensurable across tasks, so the primary aggregate throughout is normalized retention,

$$\text{NormRet}_t = \frac{\text{Score}_t(\theta_{\text{merge}}) - \text{Score}_t(\theta_0)}{\text{Score}_t(\theta_t) - \text{Score}_t(\theta_0) + \epsilon_{\text{metric}}}, \quad (8)$$

which places the pretrained model at 0 and the task expert at 1 (ϵ_{metric} guards small denominators). Mean and worst-task values are always reported together—worst-task matters because a merge can improve the average while catastrophically harming a small task. Two benchmarks depart from this convention because their retention denominators degenerate: on the twenty-task CLIP suite several experts barely improve on the zero-shot backbone, and on MergeBench the multilingual expert finishes *below* the pretrained model (0.494 vs. 0.495), so a near-zero or negative denominator makes per-task retention explode or invert. Both benchmarks therefore report absolute mean and worst-task accuracy, quoting the pretrained floor and expert ceiling for scale (Section 5.1).

Protocol. Refinement runs S sweeps from the given initialization with the trust region applied after the learning rate (Eq. (6)), $\gamma = 1$, $\epsilon_{\text{gate}} = 0$, a constant learning rate, and a fixed task order. Each family—ordinary replay gradient descent (GD), the proposed gated update (APR), and the ungated distance-scaled update—receives an equal-size search over the learning rate and the sweep count S . Selection never reads the evaluation split: each task contributes a labeled replay buffer used for the refinement sweeps and a *disjoint* selection buffer of the same size, every hyperparameter—including each merge baseline’s own coefficient and density—is chosen by minimizing loss on the selection buffer, and the evaluation split is read only at the selected cell. The main grid uses 16 examples per task in each buffer; the budget study of Section 5.1 pushes the refinement buffer to 8 and replicates the selected cells over pinned replay draws, so that the reported variance isolates the draw, and the MergeBench track runs at 8+8 throughout, generation-based evaluation being the dominant cost there. The multi-head oracle-task-id assumption is removed on both the MergeBench and CLIP tracks, whose merged models carry no task-specific parameters.

Table 1: **The experiment grid under split selection at a 16+16 budget**, instantiated identically on every encoder benchmark. Each task contributes 16 labeled replay examples for the refinement sweeps and a *disjoint* 16-example selection buffer (32 labels per task in total). Every hyperparameter—merge coefficient, density, refinement learning rate, and the sweep count S —is chosen by minimizing held-out loss on the selection buffer, and the evaluation split is read *only* at the selected cell, so the table reports what a practitioner obtains without ever touching test data. Entries are mean aggregate with worst-task in parentheses: normalized retention on GLUE-8 and CLIP-8, where the pretrained *alone* row is the zero point, and absolute top-1 accuracy on twenty-task CLIP, whose retention denominator degenerates (Section 4). AdaMerging is given the matched budget—the same 16 unlabeled inputs per task that APR sees labeled—rather than the standard transductive formulation that adapts on the evaluation split. The horizon grid is $S \in \{5, 20, 50\}$ on CLIP-20 and on the merge-initialized GLUE-8 rows, and $S \in \{5, 20, 50, 100, 200\}$ on CLIP-8 and the from-pretrained GLUE-8 rows; every arm receives the same grid as the arms beside it, so no treatment is searched over a wider space than its controls. $\dagger$: the selected S is the longest its grid offered, so the cell is a lower bound rather than a verified interior optimum—at twenty tasks and from a merge initialization the refinement is still improving at $S = 200$. The decoder-only MergeBench track runs at a smaller budget and is reported separately in Table 2.

Initialization	Treatment	GLUE-8 normalized retention	CLIP-8 normalized retention	CLIP-20 top-1 acc.
1 0.95	Pretrained	alone	0	0.550 (+.10)
		+GD	0.209 (−.18)	0.583 $\dagger$ (+.12)
		+ungated	−0.018 (−.49)	0.586 (+.11)
		+APR	0.315 (−.12)	0.647 (+.22)
	Task arithmetic	alone	0.320 (−.18)	0.495 (+.15)
		+GD	0.574 (+.06)	0.555 (+.18)
		+ungated	0.586 (−.01)	0.602 (+.17)
		+APR	0.608 (+.10)	0.667 (+.25)
	Breadcrumbs	alone	0.030 (−2.14)	0.603 (+.12)
		+GD	0.534 (+.04)	0.638 $\dagger$ (+.17)
		+ungated	0.563 (+.04)	0.650 (+.19)
		+APR	0.567 (+.04)	0.661 (+.20)
	DARE-TIES	alone	0.301 (−.34)	0.520 (+.20)
		+GD	0.521 (−.08)	0.523 $\dagger$ (−.07)
		+ungated	0.574 (+.09)	0.291 $\dagger$ (−1.85)
		+APR	0.586 (+.11)	0.579 (+.09)
			0.527 $\dagger$ (−.39)	0.664 (+.30)
	TIES	alone	0.208 (−.22)	0.619 (+.19)
		+GD	0.516 (−.13)	0.600 $\dagger$ (+.18)
		+ungated	0.525 (−.08)	0.561 (−.20)
		+APR	0.543 (−.03)	0.608 $\dagger$ (+.11)
				0.676 (+.27)
	AdaMerging	alone	0.029 (−3.02)	0.533 (+.09)
		+GD	0.459 $\dagger$ (−.35)	0.597 (+.10)
		+ungated	0.601 $\dagger$ (+.14)	0.637 $\dagger$ (+.13)
			0.538 (−.36)	0.649 (+.16)
		+APR	0.557 $\dagger$ (−.04)	0.649 (+.16)
			0.564 (−.02)	0.665 (+.20)
			0.638 $\dagger$ (+.32)	

5 RESULTS

Table 1 instantiates one experiment grid identically on every benchmark, under a selection protocol that never reads the evaluation split; Section 5.1 then pushes the labeled budget lower still on the benchmark where interference is worst. Per-benchmark details, diagnostics, and the full grids are given in Appendix D.

The grid under split selection. Table 1 reports the three encoder benchmarks at a labeled budget of 16 refinement and 16 selection examples per task; Table 2 repeats the two eight-task encoder benchmarks at half that budget and adds the decoder-only MergeBench track, which runs at 8+8 throughout. In both tables no evaluation data is used for tuning. The result splits cleanly in two, along the axis of whether a usable merge already exists.

Refining *from the pretrained model*—the setting with no merge to inherit, and the one where the refinement must supply all of the multi-task ability itself—is an unambiguous win for the gated, expert-anchored update. APR is the best treatment on all three encoder benchmarks, ahead of the

Table 2: **The canonical grid under split selection at an 8+8 budget.** Each task contributes eight labeled replay examples and a disjoint eight-example selection buffer. Every hyperparameter is selected on the selection buffer before the evaluation split is read; the encoder tracks use $S \in \{5, 20, 50\}$ and MergeBench uses $S \in \{5, 20, 50, 100\}$. Entries follow Table 1: mean normalized retention with worst-task retention in parentheses, except MergeBench and CLIP-20, which report absolute accuracy. The decoder-only MergeBench track uses Llama-3.2-3B with mathematics, coding, instruction-following, and multilingual experts; its pretrained floor is 0.289 and expert ceiling 0.548. $\dagger$ marks a selected horizon at the longest value offered; dashes are configurations not run.

Initialization	Treatment	GLUE-8 normalized retention	CLIP-8 normalized retention	MergeBench absolute accuracy	CLIP-20 absolute accuracy
Pretrained	alone	0	0	0.289 (.06)	—
	+GD	−0.361 (−3.52)	−0.148 (−1.04)	0.333 (.15)	—
	+ungated	0.089 † (−.31)	−0.006 (−.41)	—	—
	+APR	0.223 † (−.33)	0.295 † (−.45)	0.337 (.20)	—
Task arithmetic	alone	0.320 (−.18)	0.270 (−.53)	—	—
	+GD	0.530 (+.04)	0.167 † (−1.10)	—	—
	+ungated	0.529 (+.03)	0.241 † (−.79)	—	—
	+APR	0.498 (−.18)	0.462 † (−.12)	—	—
Breadcrumbs	alone	0.030 (−2.14)	0.431 (+.08)	—	—
	+GD	0.512 (−.00)	0.479 † (+.04)	—	—
	+ungated	0.536 (+.05)	0.391 † (−.40)	—	—
	+APR	0.528 (−.06)	0.339 † (−.46)	—	—
1 0.95 DARE-TIES	alone	0.301 (−.34)	0.364 (−.32)	—	—
	+GD	0.484 (−.08)	−1.273 † (−4.87)	—	—
	+ungated	0.534 (+.08)	−0.578 (−3.11)	—	—
	+APR	0.576 (+.19)	0.498 † (−.31)	—	—
TIES	alone	0.208 (−.22)	0.525 (+.21)	—	—
	+GD	0.442 (−.10)	0.585† (+.18)	—	—
	+ungated	0.442 (−.07)	0.318 † (−1.57)	—	—
	+APR	0.489 † (+.01)	0.566 † (−.08)	—	—
AdaMerging	alone	0.028 (−3.01)	0.463 (−.04)	—	—
	+GD	0.016 (−2.95)	0.502 † (−.06)	—	—
	+ungated	0.466 (−.10)	0.285 † (−1.41)	—	—
	+APR	0.543 (+.02)	0.554 † (+.15)	—	—
RegMean	alone	—	—	—	—
	+GD	—	—	—	—
	+ungated	—	—	—	—
	+APR	—	—	—	—

better of the two unconstrained baselines by 0.06 (CLIP-20), 0.11 (GLUE-8), and 0.39 (CLIP-8); the decoder track of Table 2, where the margins are carried by the worst task, is discussed below. Neither unconstrained baseline is a viable cold-start method at this budget. Ordinary descent recovers between a twentieth and a fifth of the expert–base gap on the two retention benchmarks (0.046 on CLIP-8, 0.209 on GLUE-8), and it buys even that by running to the end of its horizon grid. The ungated update fares worse still: given the same long horizons it fails to improve on the pretrained model at all on either benchmark (−0.018 on GLUE-8, −0.036 on CLIP-8), so distance scaling without the gate is not merely weaker than APR but useless from this initialization. Only the gated, distance-scaled update turns a small replay buffer into multi-task ability with no merge to build on.

The decoder track. On MergeBench (Table 2), both labeled arms improve the pretrained Llama-3.2-3B at eight labels per task—mean accuracy 0.289 $\rightarrow$ 0.333 (GD) and $\rightarrow$ 0.337 (APR) against an expert ceiling of 0.548—and the difference between them is carried almost entirely by the worst task: APR holds 0.200 on mathematics, the task the pretrained model is weakest at, against GD’s 0.150, while the mean gap (+0.004) is within evaluation noise on 300-item sets. The degenerate multilingual expert is instructive twice over. It forces the absolute-accuracy convention (its negative expert–base gap would invert normalized retention), and the refinement diagnoses it independently: the attribution gate passes only 16% of coordinates for the multilingual task against 38–46% for the other three—the sign test finds little locally loss-decreasing structure in an expert that has nothing to teach, exactly as its evaluation scores indicate. The track is single-seed, and the ungated arm and composition rows have not been run, so the decoder evidence is thinner than the encoder tracks’.

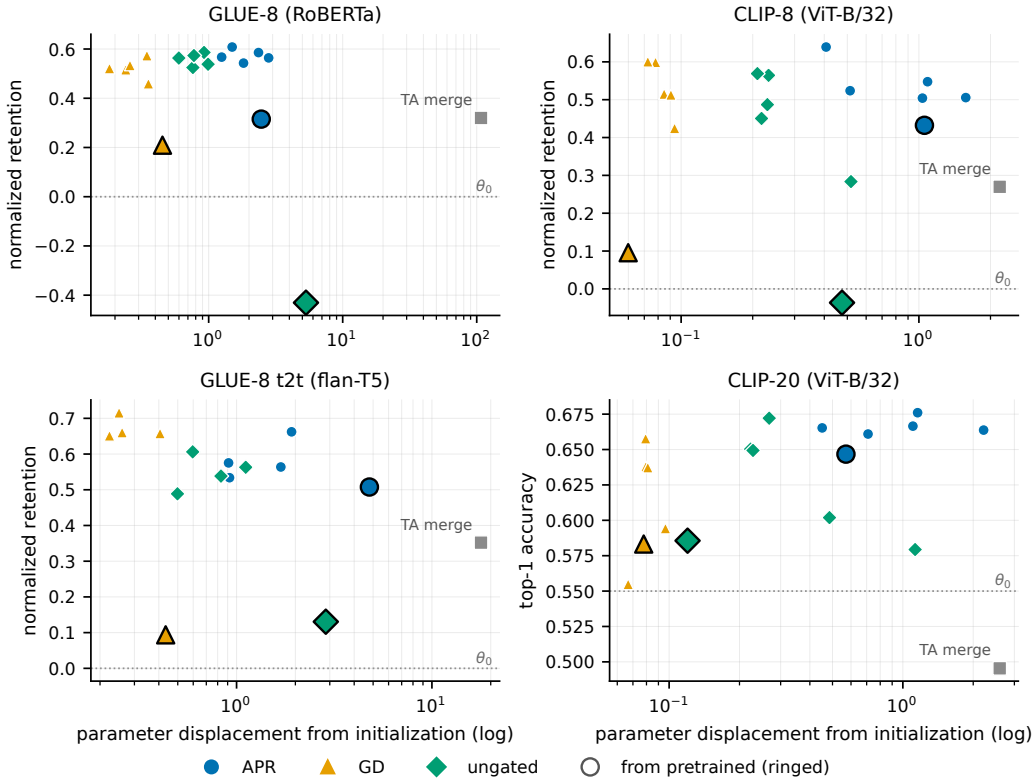


Figure 1: Score versus parameter displacement for every split-selected refinement cell of Table 1; log x . Ringed markers refine from the pretrained model, so their displacement is the distance from θ_0 itself; small markers refine from a merge initialization and show the distance from that initialization; the gray square is the best task-arithmetic merge at its distance from θ_0 (task arithmetic is linear in λ , so this follows from one measured norm). Two regularities hold on every benchmark. Refinement is a small-displacement operation relative to merging—on GLUE-8 the tuned merge sits ≈ 108 from θ_0 for retention 0.320, while APR from θ_0 reaches 0.315 at displacement 2.5, matching it at $\approx 2\%$ of the parameter drift. And movement separates from harm by arm: split selection keeps ordinary GD close to its start, where from θ_0 it achieves little, whereas the ungated update both travels far and can end catastrophically (GLUE-8, -0.43 at displacement 5.3); APR is the arm that converts displacement into score. Parameter distance is a within-family diagnostic rather than a capability measure: it is comparable along the task-arithmetic family, whose merges differ only by a scalar, but not across merge families with different geometry (RegMean, in particular, sits over 450 from θ_0 in directions that are largely inference-invisible; Appendix D). Whether the small displacements on this figure correspond to preserved pretrained capability is therefore measured rather than inferred, in Table 3.

Does small displacement preserve pretrained capability? Figure 1 shows the refinement reaching merge-level scores at a small fraction of the merge’s displacement, but parameter distance is a geometric quantity, not a capability measure. We therefore probe the base model’s *own* objective directly: the masked-language-model loss of RoBERTa’s pretrained LM head—left untouched, so the only thing varying is the trunk—evaluated on WikiText-2 validation text that no part of the experiment ever sees. The probed states are exactly the split-selected from-pretrained cells of Table 1, re-derived from the same replay buffer, with the pretrained model and the task-arithmetic merge as references (Table 3).

Three things follow. First, the proximity story survives a functional test: APR matches the merge’s GLUE-8 retention (0.315 vs. 0.320) while inflating the pretraining loss by 1.08 against the merge’s 2.51—it buys the same multitask ability for 43% of the damage to what the model already knew. Second, and less expected, displacement does not order the drift even *within* this tier: APR travels $5.5\times$ farther from θ_0 than ordinary descent, yet ends closer to the base model in function ($+1.08$

Table 3: Pretraining-capability drift on multi-head GLUE-8 at the 16+16 budget. Each refinement row is the split-selected from-pretrained cell of Table 1, re-derived from the same replay buffer; “MLM loss” is RoBERTa’s own pretrained masked-LM head evaluated over the refined trunk on WikiText-2 validation, text no experiment touches, so the increase over θ_0 is pure trunk drift measured in the base model’s own objective. Single seed. Lower MLM loss and higher retention are both better; APR is the only arm that improves on ordinary descent in both.

State (from θ_0)	Dist. θ_0	GLUE-8 NormRet	MLM loss (Δ)
Pretrained θ_0	0.00	0	2.51 (0.00)
Task-arithmetic merge ($\lambda=0.3$)	107.7	0.320	5.02 (+2.51)
+GD ($\eta=10^{-3}$, $S=50$)	0.45	0.209	4.13 (+1.62)
+ungated ($\eta=8$, $S=50$)	5.32	−0.431	16.10 (+13.59)
+APR ($\eta=8$, $S=50$)	2.47	0.315	3.59 (+1.08)

vs. +1.62), so APR dominates GD on both axes at once. What the anchor buys is not a shorter path but a better-directed one. Third, the gate is doing the protecting here rather than the anchor: the ungated update, identical except for the gate and run at the same rate, does not merely score badly (−0.431) but destroys the pretraining objective outright (16.10 against a base of 2.51). The probe is a single seed on a single corpus for one benchmark, and it measures the pretraining objective rather than downstream ability; the multi-task version of this question—retention on held-out *tasks*—is the subject of Section 5.1.

5.1 TWENTY-TASK CLIP (ViT-B/32)

The twenty-task suite of Wang et al. (2024) (published fine-tuned ViT-B/32 encoders) is the high-interference end of the vision track, and the benchmark on which Table 1 shows the widest gap between the anchored and unconstrained arms. The scale forces two protocol changes. The conventional $\lambda_i = 0.3$ is catastrophic here—the task-arithmetic merge scores 0.364, *below* the 0.550 zero-shot floor, and the tuned optimum falls to $\lambda \approx 0.05$ –0.08—so merge coefficients must be re-tuned per task count rather than inherited. And, as declared in Section 4, normalized retention degenerates on near-zero expert–base gaps (STL10’s expert beats zero-shot by 0.004), so this section reports absolute mean and worst-task top-1 accuracy (zero-shot floor 0.550, expert ceiling 0.896).

Down to eight labels per task. We push the budget below the grid’s 16 examples per task with a full $n \in \{8, 16, 32\}$ grid, each cell tuned over the learning rate and horizon $S \in \{5, 30, 60\}$ (Table 5, Appendix D). Three regularities. First, degradation with budget is graceful in the mean but concentrated in the *worst task*: from AdaMerging, mean accuracy loses 0.015 across the $4\times$ budget reduction ($0.698 \rightarrow 0.683$) while worst-task accuracy loses 0.034. Second, the refinement’s value survives at the smallest budget: with just *eight* labeled examples per task, refining the pretrained model reaches 0.632—above the tuned task-arithmetic merge (0.604), which uses no labels but requires all twenty expert checkpoints and a tuned coefficient—and that holds over five replay draws (0.626 ± 0.005 , above the merge on every draw). Third, from the pretrained model the anchored update’s margin over plain descent is large and essentially budget-independent ($\approx +0.05$ –0.06 mean at every n ; at $n = 8$, 0.626 ± 0.005 against 0.572 ± 0.003 over five draws). Geometry, not data volume, sets that gap: at the tuned merge every expert sits ≈ 2.4 away while the best refined solutions lie 0.14–0.23 from a strong initialization, so the distance-scaled step matters most exactly where the destination genuinely is far—from the pretrained model (Appendix D).

The anchored step, not the gate, is load-bearing. APR and the ungated anchored update agree within ± 0.01 across every initialization tested on this suite, while the inverted gate collapses to 0.100: the attribution *sign* carries signal, but thresholding at zero adds nothing over no gate. A held-out audit explains why, and closes the route of thresholding harder: first-order attributions transfer to a disjoint buffer only weakly— $\approx 10\%$ of the predicted loss decrease at $n = 8$, rising to only $\approx 32\%$ even at a four-times larger buffer—and uniformly across magnitude buckets, so a top- k rule would discard most of the transferable signal; at $n = 8$ the ordering across buckets even reverses, the largest gradient entries being the most sample-specific. The gradient’s transferable contribution is step magnitude, not coordinate choice (Appendix D).

Safety at small budgets. Safety is the method’s clearest practical advantage, and it is sharpest exactly where the buffer is small. Best-per-cell tables cannot detect overfitting, since selected configurations exclude it by construction, so we audit *every* (η, S, n) cell for the signature of replay loss improving while test accuracy falls. For unconstrained descent it appears as soon as the buffer is small: 9 of 36 small- n cells deteriorate, with textbook cases at $n = 16$ (replay loss $0.083 \rightarrow 0.063$ while test accuracy falls $0.636 \rightarrow 0.288$), plus two outright instabilities. For the anchored update the signature never appears—across ≈ 48 cells the largest decline is 0.003 —*even though* APR also reaches near-interpolation of its buffer (replay loss 0.007 – 0.011 at $n = 8$). Once every coordinate is capped at $\gamma|v_{i,r}|$, additional sweeps have nowhere damaging to go, whereas descent keeps moving after it has fit the buffer; a corollary is that descent’s safe learning-rate window narrows as sweeps accumulate, so a rate tuned at $S = 5$ cannot be reused at $S = 60$, while APR’s optimal rate transfers unchanged across horizons (Appendix D).

Held-out-task retention: preserved capability, measured. Table 3 probed the pretraining objective; here we measure retained *ability*. We split the suite into the 14 specialized training tasks and the 6 natural-image tasks on which the base model’s zero-shot accuracy is highest (mean 0.802 , so there is something to lose): every merge and every refinement step sees only the 14 training tasks and their 16-example buffers, and the held-out tasks are evaluated purely zero-shot through frozen text-derived heads, so nothing task-specific leaks. This is a frontier measurement, not a selected cell—each arm is swept over its learning rate and the whole arc is reported—so it is compatible with the split-selection protocol without a selection buffer: nothing is being selected. Table 4 and Figure 2 report the outcome. **Every checkpoint-only merge is dominated:** task arithmetic at all four coefficients, TIES at both, DARE-TIES, and Breadcrumbs each have a refinement point above them on both axes, and AdaMerging is dominated by APR launched from it (0.648 train at -0.018 against 0.614 at -0.026). APR holds three of the frontier’s four non-trivial points; the extreme-accuracy end belongs to RegMean (0.692 at -0.041), the one merge family the refinement does not dominate here. Both unconstrained controls are dominated by a single APR point: at the *same* learning rate, the ungated update gives up 0.051 training accuracy *and* 0.044 held-out accuracy to the gated one ($0.500/-0.050$ vs. $0.551/-0.006$)—the gate protects the shared features that zero-shot transfer depends on, the same conclusion the drift probe reached in the base model’s own objective—and ordinary descent lands at $0.491/-0.016$, worse than APR at $\eta = 2$ on both axes despite moving a third as far. Two cautions transfer from the geometry analysis: parameter distance fails as a drift proxy across merge families (RegMean sits 227 from θ_0 yet loses only 0.041 held-out, while task arithmetic at distance 2.2 loses 0.208), and tuning the merge coefficient on the merged tasks alone is actively harmful ($\lambda=0.08 \rightarrow 0.2$ loses on both axes). The study is single-seed at this budget.

Summary. Five claims survive the campaign at $n \leq 16$: (i) the refinement improves every initialization it is given—five merges and the pretrained model—with gains shrinking but never vanishing as the initialization improves; (ii) the anchored, distance-scaled, clipped step is the load-bearing ingredient, the gate contributing worst-task repair, small-budget stability, and held-out retention rather than mean accuracy; (iii) no anchored run fell below its initialization or the zero-shot floor at any rate, horizon, or budget tested, and none showed the overfitting signature, while unconstrained descent deteriorates on a quarter of its small-buffer configurations with its replay loss still improving; (iv) eight labels per task suffice to beat the tuned task-arithmetic merge starting from the pretrained model alone; and (v) on held-out tasks never merged or refined, the refinement populates the accuracy–retention frontier and every checkpoint-only merge is dominated (Figure 2).

6 DISCUSSION

This work develops an attribution-patching perspective on replay-guided merging: the sign of $g_{i,r}v_{i,r}$ tells, per coordinate, whether moving toward expert i locally reduces replay loss; the same condition is a trust-region acceptance rule, and both views motivate one procedure—mask, distance-scale, clip, apply task-sequentially. Evaluated under a protocol that never reads the evaluation split, at 16 labeled examples per task or fewer, the refinement is a *complement* to merging: it improves every initialization tried, and its outcome is markedly less sensitive than unconstrained descent’s to which merge it was handed. Its decisive setting is the one with no merge to inherit—from the pretrained model the gated, expert-anchored update is the best treatment on every benchmark reported, while neither unconstrained baseline is viable at this budget—and its advantage grows as the buffer shrinks,

Model (built from 14 train tasks)	Dist. θ_0	Train-14	Held-out 6 (drop)
Base model θ_0^*	0.00	0.443	0.802 (0.000)
GD from base, $\eta=5e-5$	0.09	0.491	0.786 (−0.016)
APR from base, $\eta=2^*$	0.31	0.551	0.796 (−0.006)
APR from base, $\eta=4^*$	0.42	0.564	0.790 (−0.012)
APR from base, $\eta=8$	0.59	0.592	0.779 (−0.023)
Ungated from base, $\eta=2$	0.65	0.500	0.752 (−0.050)
Task arith. ($\lambda=0.08$)	0.87	0.536	0.781 (−0.021)
APR from task arith., $\eta=4$	1.05	0.610	0.772 (−0.029)
Breadcrumbs	1.28	0.536	0.792 (−0.010)
APR from AdaMerging, $\eta=4^*$	1.43	0.648	0.784 (−0.018)
AdaMerging-layer	1.69	0.614	0.776 (−0.026)
Task arith. ($\lambda=0.2$)	2.18	0.510	0.594 (−0.208)
TIES	2.90	0.578	0.770 (−0.032)
RegMean*	227	0.692	0.761 (−0.041)

Table 4: Held-out-task retention on the twenty-task suite at the 16-example budget: merge/refine on 14 specialized tasks, evaluate the 6 strongest natural-image tasks zero-shot (base mean 0.802); single seed, refinements at $S=30$ cosine. Full learning-rate arcs are reported—nothing is selected on the evaluation split. * marks the accuracy/retention Pareto frontier; non-frontier rows abridged.

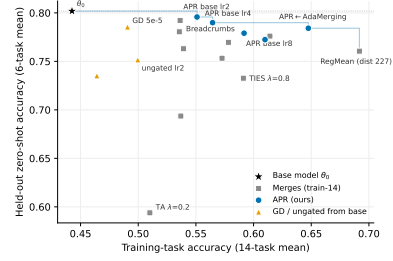


Figure 2: The accuracy–retention trade-off of Table 4: models built from the 14 training tasks, evaluated on them (x) and zero-shot on the 6 held-out tasks (y ; dotted line: the base model). The frontier is populated by the anchored refinement and RegMean; every checkpoint-only merge (gray squares) and both unconstrained controls (triangles) are dominated.

where descent overfits while its replay loss still improves and the trust region removes that failure mode outright. Refining from the pretrained model also turns out to be cheap in what it costs the pretrained model: it reaches merge-level multitask retention while inflating the base objective’s loss by well under half of what the merge does, and it does so while moving *farther* in parameter space than ordinary descent, which drifts more—so the anchor buys a better-directed path rather than a shorter one. The same conclusion holds one level up, on retained *ability* rather than the pretraining objective: on tasks never merged or refined, the refinement traces the accuracy–retention frontier, with the gate—not just the anchor—supplying the retention. The result is a lightweight bridge between checkpoint-only task arithmetic and full multitask replay fine-tuning, usable at a labeled budget of a few dozen examples per task.

AI USE STATEMENT

In this work, we used generative AI tools to assist with writing and refactoring experiment code and with editing and polishing the text. We did not use generative AI tools for research ideation, experimental design, or data analysis. We have reviewed all AI-assisted work and take responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of generative AI.

REFERENCES

- Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.
- Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.
- Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.
- Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. MergeBench: A benchmark for merging domain-specialized LLMs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.

- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.
- Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.
- Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.
- Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.
- Wenju Sun, Qingyong Li, Wen Wang, Yangli-ao Geng, and Boyang Li. Task Arithmetic in Trust Region: A training-free model merging approach to navigate knowledge conflicts. *arXiv preprint arXiv:2501.15065*, 2025.
- Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.
- Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.
- Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jiménez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. In *International Conference on Machine Learning*, 2024.
- Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.
- Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.
- Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.

A BOX-DISTANCE MONOTONICITY AND THE EXPERT BOX

This appendix proves the general geometric result behind Corollary 1. The statement is self-contained: $x_1, \dots, x_n$ are arbitrary points in $\mathbb{R}^D$ (in the application, the task experts $\theta_1, \dots, \theta_T$), and the superscript d indexes coordinates.

Theorem 1 (Box-distance monotonicity under coordinatewise movement toward a hull point). *Let $x_1, \dots, x_n \in \mathbb{R}^D$. Define the axis-aligned box hull*

$$B := \prod_{d=1}^D [m_d, M_d], \quad m_d := \min_{1 \leq j \leq n} x_j^d, \quad M_d := \max_{1 \leq j \leq n} x_j^d.$$

Let $y, z \in \mathbb{R}^D$. Suppose that there exists some $i \in \{1, \dots, n\}$ such that, for every coordinate $d \in \{1, \dots, D\}$, the point z^d lies between x_i^d and y^d . Equivalently,

$$|z^d - x_i^d| \leq |y^d - x_i^d|$$

and $z^d - x_i^d$ has the same sign as $y^d - x_i^d$, allowing equality.

Then, for every $1 \leq p \leq \infty$,

$$d_p(z, B) \leq d_p(y, B),$$

where $d_p(u, B)$ denotes the distance from u to B with respect to the ℓ^p -norm.

Proof. For each coordinate d , set

$$I_d := [m_d, M_d].$$

Since $x_i \in B$, we have

$$x_i^d \in I_d$$

for every d .

Define the one-dimensional distance function

$$\delta_d(t) := \text{dist}(t, I_d) = \inf_{s \in I_d} |t - s|.$$

We first prove the following elementary one-dimensional claim.

Claim. Let $I = [m, M] \subset \mathbb{R}$, let $c \in I$, and suppose that z lies between c and y . Then

$$\text{dist}(z, I) \leq \text{dist}(y, I).$$

Indeed, if $y \in I$, then since $c \in I$ and I is an interval, every point between c and y also lies in I . Hence $z \in I$, so

$$\text{dist}(z, I) = 0 = \text{dist}(y, I).$$

If $y > M$, then

$$\text{dist}(y, I) = y - M.$$

Since z lies between $c \in I$ and $y > M$, we have $z \leq y$. If $z \leq M$, then $\text{dist}(z, I) = 0$. If $z > M$, then

$$\text{dist}(z, I) = z - M \leq y - M = \text{dist}(y, I).$$

The case $y < m$ is symmetric. This proves the claim.

Now apply the claim coordinatewise with

$$I = I_d, \quad c = x_i^d.$$

Because z^d lies between x_i^d and y^d , we obtain

$$\delta_d(z^d) \leq \delta_d(y^d)$$

for every $d \in \{1, \dots, D\}$.

Next, distance to an axis-aligned box separates coordinatewise. Namely, for $1 \leq p < \infty$,

$$d_p(u, B) = \left(\sum_{d=1}^D \delta_d(u^d)^p \right)^{1/p},$$

and for $p = \infty$,

$$d_\infty(u, B) = \max_{1 \leq d \leq D} \delta_d(u^d).$$

This follows because the closest point of B to u is obtained by clipping each coordinate u^d to the interval $I_d = [m_d, M_d]$.

Since

$$\delta_d(z^d) \leq \delta_d(y^d)$$

for every d , monotonicity of the ℓ^p -norm on the nonnegative orthant gives

$$d_p(z, B) \leq d_p(y, B).$$

This proves the theorem. $\square$

The box hull B may look like a loose relaxation of the ordinary convex hull of the experts, but it is canonical in its own right: it is the smallest set containing the experts that is convex with respect to ℓ^1 -geodesics.

Proposition 2. *The set B is the ℓ^1 -geodesic convex hull of $\{x_1, \dots, x_n\}$.*

Proof. For $a, b \in \mathbb{R}^D$, define the ℓ^1 -metric interval between a and b by

$$I_1(a, b) := \{u \in \mathbb{R}^D : \|a - u\|_1 + \|u - b\|_1 = \|a - b\|_1\}.$$

We claim that

$$I_1(a, b) = \prod_{d=1}^D [\min(a^d, b^d), \max(a^d, b^d)].$$

Indeed, by the triangle inequality in one dimension,

$$|a^d - u^d| + |u^d - b^d| \geq |a^d - b^d|$$

for each coordinate d . Summing over d , equality in the ℓ^1 -triangle inequality holds if and only if equality holds in every coordinate. In one dimension, equality holds precisely when u^d lies between a^d and b^d . Hence the displayed formula for $I_1(a, b)$ follows.

Now B is clearly ℓ^1 -geodesically convex: if $a, b \in B$, then every point coordinatewise between a and b also lies in B .

It remains to show minimality. Let $C \subseteq \mathbb{R}^D$ be any ℓ^1 -geodesically convex set containing $x_1, \dots, x_n$. Since C is closed under ℓ^1 -metric intervals, it is closed under taking coordinatewise boxes between pairs of its points. In particular, if $a, b \in C$, then both the coordinatewise minimum

$$a \wedge b := (\min(a^1, b^1), \dots, \min(a^D, b^D))$$

and the coordinatewise maximum

$$a \vee b := (\max(a^1, b^1), \dots, \max(a^D, b^D))$$

belong to C .

By induction, C therefore contains the coordinatewise minimum

$$m := (m_1, \dots, m_D)$$

and the coordinatewise maximum

$$M := (M_1, \dots, M_D)$$

of the points $x_1, \dots, x_n$. Since C is ℓ^1 -geodesically convex, it contains the entire ℓ^1 -metric interval between m and M . But

$$I_1(m, M) = \prod_{d=1}^D [m_d, M_d] = B.$$

Therefore every ℓ^1 -geodesically convex set containing $x_1, \dots, x_n$ also contains B . Hence B is the smallest such set, i.e.

$$B = \text{gconv}_{\ell^1} \{x_1, \dots, x_n\}.$$

□

We can now state and prove the guarantee for Algorithm 1 used in Section 3.2.

Corollary 1 (Box-distance monotonicity of the sequential refinement). *Let*

$$B_{\text{exp}} := \prod_{r=1}^d [\underline{\theta}_r, \bar{\theta}_r], \quad \underline{\theta}_r := \min_{j \in \mathcal{T}} \theta_{j,r}, \quad \bar{\theta}_r := \max_{j \in \mathcal{T}} \theta_{j,r},$$

be the axis-aligned box hull of the task experts. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every $1 \leq p \leq \infty$,

$$d_p(\theta^{(s,k)}, B_{\text{exp}}) \leq d_p(\theta^{(s,k-1)}, B_{\text{exp}}).$$

Consequently, after any number S of sweeps,

$$d_p(\theta^{(S)}, B_{\text{exp}}) \leq d_p(\theta^{(0)}, B_{\text{exp}}) \quad \text{for every } 1 \leq p \leq \infty.$$

Proof. Fix a sweep s , a within-sweep index k , and let $i = \pi_s(k)$. We first verify the hypothesis of Theorem 1 with

$$y = \theta^{(s,k-1)}, \quad z = \theta^{(s,k)}, \quad x_i = \theta_i.$$

It is enough to check that, for every coordinate r , the new coordinate $\theta_r^{(s,k)}$ lies between the previous coordinate $\theta_r^{(s,k-1)}$ and the expert coordinate $\theta_{i,r}$.

Write $v_r = v_{i,r}^{(s,k)} = \theta_{i,r} - \theta_r^{(s,k-1)}$. If the gate is off, then $m_{i,r}^{(s,k)} = 0$, hence $u_{i,r}^{(s,k)} = 0$, and the coordinate is unchanged. If the gate is on, then

$$g_{i,r}^{(s,k)} v_r < -\epsilon_{\text{gate}} \leq 0,$$

so $-g_{i,r}^{(s,k)}$ has the same sign as v_r . Therefore the raw update $\tilde{u}_{i,r}^{(s,k)} = -g_{i,r}^{(s,k)} |v_r| m_{i,r}^{(s,k)}$ points in the same direction as v_r . Multiplication by $\eta \geq 0$ preserves this direction, possibly making the update zero, and the symmetric clip in Equation (6) gives an applied update with the same weak sign as v_r and magnitude at most $|v_r|$. Thus there is an $\alpha_{i,r}^{(s,k)} \in [0, 1]$ such that

$$u_{i,r}^{(s,k)} = \alpha_{i,r}^{(s,k)} v_{i,r}^{(s,k)}.$$

The same representation is assumed directly in the optional post-processing case. Hence

$$\theta_r^{(s,k)} = \theta_r^{(s,k-1)} + \alpha_{i,r}^{(s,k)} (\theta_{i,r} - \theta_r^{(s,k-1)}),$$

so $\theta_r^{(s,k)}$ lies between $\theta_r^{(s,k-1)}$ and $\theta_{i,r}$.

The box B_{exp} is exactly the axis-aligned hull of the expert points $\theta_1, \dots, \theta_T$. Therefore Theorem 1 gives

$$d_p(\theta^{(s,k)}, B_{\text{exp}}) \leq d_p(\theta^{(s,k-1)}, B_{\text{exp}})$$

for every $1 \leq p \leq \infty$. Chaining this inequality over $k = 1, \dots, T$ and over sweeps $s = 0, \dots, S - 1$ gives the final statement. □

B METHOD DETAILS: SEQUENTIAL SCHEDULING

This appendix details the scheduling of the per-task updates, summarized in Section 3.2.

A secondary design choice concerns how the per-task updates are scheduled within a refinement sweep. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task’s update immediately after computing it. This has two motivations. First, each update is applied at the same local model state on which its gradient, expert direction, and gate were computed. Second, the trust-region clipping in Equation (6) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step.

The clip bound $|v_{i,r}^{(s,k)}|$ may optionally be scaled by a factor γ ($\gamma = 1$ throughout the paper). Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $\gamma |v_{i,r}^{(s,k)}|$ for any η ; for $\gamma \leq 1$ a single update can never overshoot the expert in a coordinate, so the refinement stays within the expert-distance trust region no matter how large η is, and the learning rate controls only how many coordinates are driven to the trust-region boundary. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j ; we therefore treat task ordering, cross-task interference, and worst-task retention as explicit diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update.

C EXTENDED EXPERIMENTAL PROTOCOL

This appendix expands the protocol of Section 4: benchmark positioning, secondary tracks, replay sampling, and the full baseline roster.

Benchmark settings. The multi-head GLUE-8 benchmark (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, excluding WNLI as is common) (Wang et al., 2018) uses RoBERTa-base as the encoder backbone (Liu et al., 2019): the shared encoder is merged and evaluated with each task’s own trained head. This multi-head setup is convenient and common, but it assumes oracle task identity at evaluation time and should not be the only evidence for the method. The recent merging literature evaluates on three benchmark families—CLIP/ViT vision task-vector suites, GLUE and text-to-text NLP, and decoder-only LLM merging, the last standardized by MergeBench with released full-parameter Llama/Gemma experts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024; Wang et al., 2019; Yu et al., 2023; He et al., 2025)—and we position the proposal against all three.

Secondary benchmark settings. Two secondary tracks address the limitations of multi-head GLUE. A CLIP/ViT vision track evaluates the standard task-vector image-classification suite—SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD—testing a different modality, different loss geometry, and stronger domain shifts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024), together with its twenty-task extension (Wang et al., 2024) (adding CIFAR-10/100, STL10, Flowers102, Oxford Pets, PCAM, FER2013, EMNIST, Food101, FashionMNIST, KMNIST, RenderedSST2), which probes the high-interference regime in which merging degrades most (results in Section 5.1). A decoder-only LLM track adopts the MergeBench setting (He et al., 2025), whose published full-parameter domain experts share a common pretrained checkpoint, so task vectors and merge-to-expert directions are well defined and no expert training is required on our side. We work at a full-fine-tuning-feasible scale (Llama-3.2-3B) with the mathematics, coding, instruction-following, and multilingual experts, merging the full model including the LM head; evaluation uses judge-free automatic metrics and subsampled evaluation sets, since generation-based scoring is the dominant cost. This track removes the oracle-task-identity assumption of multi-head GLUE at a scale and modality the vision suites do not reach.

Replay data. For refinement, we use $n \in \{4, 8, 16, 32, 64, 128\}$ replay examples per task, sampled from training data or from a held-out split separated from the final validation split, and class-balanced whenever possible. Sensitivity to the replay-budget size and to the buffer draw is reported for each track, rather than only the best tuned replay size.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by data regime: *checkpoint-only* (pretrained encoder, single-task experts, uniform averaging, task arithmetic, TIES, DARE, DELLA-style sampling, Model Breadcrumbs, magnitude-based Localize-and-Stitch); *data-dependent but label-free* (RegMean, AdaMerging, APL); *labeled* (ordinary replay gradient descent from the same initialization, distance-scaled descent without the gate, head-only calibration, encoder and encoder-plus-head replay fine-tuning, multitask replay fine-tuning from the pretrained checkpoint and from the merge, Fisher merging with an empirical diagonal Fisher estimated from per-example labeled-loss gradients on the replay buffer, and Localize-and-Stitch with masks learned on the replay buffer); and *sanity controls* (inverted gate, density-matched random gate, wrong-expert directions, shuffled labels). All labeled-replay baselines use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget.

D TWENTY-TASK CLIP: EXTENDED RESULTS

This appendix gives the full twenty-task campaign behind Section 5.1.

The vision track scales to the twenty-task suite of Wang et al. (2024) using the published fine-tuned ViT-B/32 encoders for all twenty tasks (every expert was validated to beat its zero-shot floor by a wide margin, including the three tasks whose class-name orderings we had to reconstruct: PCAM $0.556 \rightarrow 0.887$, FER2013 $0.376 \rightarrow 0.711$, RenderedSST2 $0.586 \rightarrow 0.713$).

Two protocol corrections forced by the scale. First, the uniform $\lambda_i = 0.3$ that is standard at eight tasks is catastrophic at twenty: the task-arithmetic merge scores 0.364 mean top-1 accuracy, *below* the 0.550 zero-shot floor, and the tuned optimum falls to $\lambda \approx 0.05\text{--}0.08$ (accuracy 0.604; the curve is monotone over $0.3 \rightarrow 0.08$ and flat below). Merge coefficients must be re-tuned per task count, and refinement must never start from a hand-picked global λ . Second, normalized retention (Eq. (8)) becomes *degenerate* at this scale: the suite contains tasks whose expert barely improves on the zero-shot backbone (STL10 gap 0.0043, Food101 gap 0.035), so the denominator explodes—STL10 alone produces per-task retentions near -60 and is the “worst task” of nearly every method for that reason only. On this suite we therefore report *absolute* mean and worst-task top-1 accuracy (zero-shot floor 0.550, expert ceiling 0.896) as primary, with retention restricted to non-degenerate tasks as a secondary check.

Why RegMean is an awkward anchor. RegMean is the initialization at which the anchored update is least advantaged, and the reason is specific to how RegMean sits in parameter space. RegMean solves each linear layer by matching *outputs* on that layer’s inputs, $W = (\sum_i G_i)^{-1} \sum_i G_i W_i$; in input directions with negligible activation variance the solution is essentially unconstrained, so W acquires large components that are functionally inert on in-distribution inputs. The resulting merge point is over 450 from the base model in parameter norm—larger than the encoder norm itself (336)—while scoring a healthy 0.723, which is only possible because most of that displacement lies in inference-invisible directions. Every ingredient of APR reads the anchor $v_i = \theta_i - \theta$, so from this initialization v_i inherits that inflated, non-functional component: the step $-g \odot |v|$ scales noise on directions where $|v|$ is huge but the gradient is near zero, and the trust region $\gamma|v|$ is widest exactly there—which is also why the usable rate collapses by an order of magnitude here. This yields a concrete prediction: anchoring on the functional part of v (equivalently, refining RegMean from a task-vector-defined anchor) should remove the handicap—an experiment we leave to future work. It also implies that parameter-space distance from base is a valid drift proxy for the task-vector-based methods but not for RegMean, whose functional drift must be measured in output space.

Where small buffers start to overfit. A methodological point matters here: best-per-cell tables cannot detect overfitting, because selected configurations exclude it by construction, so we audit *every* (η, S, n) cell for the overfitting signature (replay loss improving from $S = 30$ to $S = 60$ while test accuracy falls). For unanchored GD the signature appears as soon as the buffer is small: 9 of 36 small- n cells deteriorate, with textbook cases at $n = 16$ (replay $0.083 \rightarrow 0.063$ while test falls $0.636 \rightarrow 0.288$; a second cell falls $0.614 \rightarrow 0.540$), plus two outright instabilities where replay worsens as well. A practical corollary is that GD’s safe learning-rate window narrows as sweeps accumulate—a rate tuned at $S = 5$ cannot be reused at $S = 60$. For the anchored update the signature never appears: across ≈ 48 cells the largest decline is 0.003, with no instabilities, *even though* APR

Table 5: Replay-budget study on the twenty-task suite: best configuration over learning rate and horizon $S \in \{5, 30, 60\}$ per cell, mean top-1 accuracy (worst-task in parentheses); single seed, identical buffer draws across cells at each n . “Init. alone” is budget-independent. Ordinary GD is reported from the base model, the setting in which the anchored step’s advantage is largest. Worst-task accuracy degrades faster than the mean as the buffer shrinks.

Initialization	Method	$n=8$	$n=16$	$n=32$
Base model (0.550 (0.100))	+ GD	0.573 (0.093)	0.580 (0.113)	0.602 (0.160)
	+ APR	0.632 (0.187)	0.640 (0.205)	0.654 (0.236)
Task arith. (0.604 (0.115))	+ APR	0.650 (0.197)	0.659 (0.242)	0.674 (0.274)
TIES (0.626 (0.150))	+ APR	0.665 (0.224)	0.673 (0.250)	0.691 (0.284)
AdaMerging (0.665 (0.101))	+ APR	0.683 (0.207)	0.688 (0.249)	0.698 (0.241)

also reaches near-interpolation (replay 0.007–0.011 on $n = 8$ buffers) and its optimal rate transfers unchanged across horizons. The absence of overfitting is therefore a property of the trust region, not of the data regime: once every coordinate is capped at $\gamma|v_{i,r}|$, additional sweeps have nowhere damaging to go, whereas GD keeps moving after it has fit the buffer.

Geometry of the twenty-task regime. Direct measurement explains why the anchor’s advantage concentrates on refinement from the pretrained model. At the tuned merge, every expert sits at distance ≈ 2.4 (encoder norm 336), but the best refined solutions found by *any* stable method lie only 0.14–0.23 from a strong initialization—about 8% of one expert distance. As tasks accumulate, the achievable joint optimum stays close to a good initialization (a compromise among twenty mutually incompatible experts, mirrored by the tuned λ falling from 0.2–0.3 to 0.08), while the expert distances $|v_i|$ that set APR’s stride stay large; the two length scales that coincided at three and eight tasks decouple at twenty. This is the mechanism behind the shrinking gate advantage and the shrinking margin over GD from a merge, and it makes a testable prediction: from the *pretrained model*, where the destination genuinely is far, the distance-scaled step should dominate small-step GD again. It does—the margin is $\approx +0.05$ –0.06 mean accuracy at every budget (Table 5), because the good region is genuinely far (> 1) while GD’s small steps travel only 0.09–0.22 and cannot reach it.

Replay budget at twenty tasks. We run a full budget grid: $n \in \{8, 16, 32\}$ labeled examples per task, each refined at $S \in \{5, 30, 60\}$ sweeps with the learning rate tuned per cell, from all four starting points, with identical buffer draws across runs (single seed). Table 5 reports the best configuration over the learning rate *and* the horizon, since the two interact: at small n nearly every best cell uses $S \geq 30$ —more passes genuinely compensate for fewer examples—and the optimal rate falls as the buffer shrinks (to $\eta \approx 1$ –4), while extending further to $S = 60$ moves best configurations by at most 0.004 at any budget, so the horizon is effectively converged by thirty sweeps. Three regularities. First, degradation with budget is graceful in the mean but concentrated in the *worst task*: from AdaMerging, mean accuracy loses 0.015 across the $4\times$ budget reduction ($0.698 \rightarrow 0.683$) while worst-task accuracy loses more than twice as much in relative terms ($0.241 \rightarrow 0.207$)—the cost of a small buffer is paid disproportionately by the hardest task, which a twenty-task mean conceals. Second, the refinement’s value survives at the smallest budget: with just *eight* labeled examples per task, refining the base model reaches 0.632 (0.626 ± 0.005 over five buffer draws, above the merge on every draw)—while the tuned task-arithmetic merge (0.604) uses no labels but requires all twenty expert checkpoints and a tuned coefficient. Third, from the base model the anchored update’s margin over plain gradient descent is large and essentially independent of the budget ($\approx +0.05$ –0.06 mean at every n ; GD row in Table 5): the geometry argument above, not data volume, sets that gap. Over five pinned buffer draws at $n = 8$ that margin is $+0.053 \pm 0.006$ (APR 0.626 ± 0.005 against GD 0.572 ± 0.003), and APR’s worst-task accuracy is higher on every draw (0.170 ± 0.017 against 0.095 ± 0.011).

Summary of the twenty-task study. Four claims survive the campaign, each with its evidence. (i) *The anchored refinement improves every initialization it is given*, including the pretrained model itself, with the gain shrinking as the initialization improves but never vanishing (Table 5, and the CLIP-20 column of Table 1). (ii) *The load-bearing ingredient is the anchored, distance-scaled, clipped step, not the attribution gate*: the gate is neutral on mean accuracy at this scale, and its thresholded

variants cannot help, because the gradient’s coordinate-level selection carries no transferable structure (the held-out audit above). (iii) *Safety is the method’s clearest practical advantage*: across every learning rate, horizon, and budget tested, no anchored-refinement run fell below its initialization or the zero-shot floor, and none showed the overfitting signature, while unconstrained descent deteriorates on a quarter of its small-buffer cells. (iv) *The method works in the small-data regime*: eight labeled examples per task suffice to beat the tuned task-arithmetic merge from the pretrained model alone, by $+0.053 \pm 0.006$ over ordinary descent across five buffer draws, with the budget’s cost concentrated in worst-task accuracy.

Limitations. Four gaps remain. The budget grid of Table 5 is single-seed, with five-seed replication only at its from-pretrained $n = 8$ cell, and the cells of Table 1 are single-draw throughout, as is the held-out retention study of Table 4 and the drift probe of Table 3. The predicted functional-anchor fix for the RegMean initialization is untested. On MergeBench, only the from-pretrained APR and GD cells were run—the ungated arm and the composition rows are absent—so the decoder track supports the cold-start comparison but not the gate-versus-anchor decomposition or the composition claims, which rest on the encoder tracks. A further limitation is one of scope rather than execution: the results reported here are confined to budgets of at most 32 labeled examples per task, so we make no claim about how the ordering between the anchored and unconstrained families behaves when labeled data is plentiful.